

18-dars — Uy vazifasi

Mavzu: **Go'da JSON bilan ishlash** (encoding/json, Marshal / Unmarshal, struct tag, fayl bilan ishlash, xatolarni log qilish)

Jami 5 ta masala · 1-4 — mashq masalalari · 5 — katta amaliy loyiha

Umumiy talablar (hamma masalalar uchun)

- Har bir masala **alohida papkada** bo'lsin: `vazifa1/`, `vazifa2/`, ... `vazifa5/`.
- Har bir papkada `main.go` fayl bo'ladi va `go run main.go` buyrug'i bilan ishga tushishi kerak.
- Kod `gofmt` bilan formatlangan bo'lsin (VS Code'da saqlaganda avtomatik bo'ladi).
- **Hech qachon xatoni e'tiborsiz qoldirmang.** Ya'ni `_ = err` deb yozmang — har bir `err != nil` tekshirilsin.
- O'zgaruvchi va funksiya nomlari ingliz tilida va mazmunli bo'lsin (`a`, `b`, `x` emas).
- Topshirish: papkalarni GitHub'ga push qiling va havolani yuboring.

Eslatma — JSON tag nima uchun kerak?

Go'da struct maydonlari katta harf bilan boshlanishi kerak (eksport bo'lishi uchun), lekin JSON faylda odatda kichik harf ishlatiladi. Shuning uchun tag yozamiz:

```
type Product struct {
    Title    string `json:"title"`           // JSON'da "title" bo'ladi
    Quantity int    `json:"qty,omitempty"`    // 0 bo'lsa JSON'ga umuman yozilmaydi
    Brand    string `json:"- "`              // JSON'ga hech qachon yozilmaydi
}
```

1-masala. Struct → JSON (Marshal) oson

Shart

`Student` nomli struct yarating. Uning ichida quyidagi maydonlar bo'lsin:

Maydon	Turi	JSON nomi	Qo'shimcha
Name	string	name	—
Age	int	age	—
Group	string	group	—
Grade	float64	grade	<code>omitempty</code> — 0 bo'lsa chiqmasin
IsActive	bool	is_active	—
Password	string	—	<code>json:"-"</code> — hech qachon chiqmasin

Bajarish kerak:

1. Bitta `Student` qiymatini to'ldiring (o'zingizning ma'lumotingiz bilan).
2. `json.Marshal` bilan JSON'ga o'giring va ekranga chiqaring.
3. Keyin `json.MarshalIndent(student, "", " ")` bilan chiroyli (chekinishli) ko'rinishda ham chiqaring.
4. Ikkalasiining farqini ko'ring va kod ostida **komment** qilib yozib qo'ying: "Marshal bir qatorda, MarshalIndent o'qishga qulay".
5. `Grade` maydonini `0` qilib ko'ring va `Password` chiqyaptimi — tekshiring.

Kutilayotgan natija (taxminan):

```
{"name": "Ali", "age": 19, "group": "Go-1", "grade": 4.5, "is_active": true}

{
  "name": "Ali",
  "age": 19,
  "group": "Go-1",
  "grade": 4.5,
  "is_active": true
}
```

Savol (kod ichida komment sifatida javob bering): `Password` nega JSON'da yo'q? `Grade` ni `0` qilganingizda nima o'zgardi?

2-masala. JSON → Struct (Unmarshal) oson

Shart

Quyidagi JSON matnini **kod ichida string** sifatida yozing:

```
const data = `{
  "name": "Iphone 15",
  "price": 1200,
  "qty": 7,
  "is_active": true,
  "category": { "id": 3, "title": "Smartfonlar" }
}`
```

Bajarish kerak:

1. Shu JSON'ga mos `Product` va `Category` structlarini yozing (ichma-ich / nested struct).
2. `json.Unmarshal([]byte(data), &product)` bilan structga o'giring.
3. Ekranga chiroyli qilib chiqaring, masalan:
`Iphone 15 (Smartfonlar) – 1200$ | qoldiq: 7 ta`
4. Endi JSON matnni **ataylab buzing** (masalan oxirgi `}` ni o'chiring) va dasturni qayta ishga tushiring. Chiqqan xatoni ekranga chiqaring.
5. `json.Valid([]byte(data))` funksiyasidan foydalanib, Unmarshal'dan **oldin** JSON to'g'riligini tekshiring. Noto'g'ri bo'lsa `"Invalid JSON"` deb yozib, dasturdan chiqing.

Muhim: `Unmarshal` ga har doim **manzil** (`&product`) beriladi. Agar `&` qo'ymasangiz struct bo'sh qoladi — buni ham sinab ko'ring.

3-masala. Slice → JSON fayl va faylni qayta o'qish oson

Shart

Kamida **5 ta** element bo'lgan `[]Book` slice yarating. `Book` struct:

```
type Book struct {
    ID      int    `json:"id"`
    Title   string `json:"title"`
    Author  string `json:"author"`
    Year    int    `json:"year"`
    Price   int    `json:"price"`
}
```

Bajarish kerak:

- `saveBooks(books []Book) error` — slice'ni `books.json` fayliga **MarshalIndent** bilan yozadigan funksiya yozing.
- `loadBooks() ([]Book, error)` — `books.json` faylini o'qib, `[]Book` qaytaradigan funksiya yozing (`os.ReadFile` yoki `os.Open + io.ReadAll`).
- `main` da: avval saqlang → keyin qayta o'qing → o'qilgan ma'lumotni tartib raqami bilan chiqaring.
- Fayl **mavjud bo'lmagan** holatni sinang: `books.json` ni o'chirib, faqat `loadBooks()` ni chaqiring. Dastur **panic bermasin**, faqat tushunarli xabar chiqarsin: `"books.json topilmadi"`.

Kutilayotgan chiqish:

- ```
1. O'tkan kunlar – Abdulla Qodiriy (1925) – 45000 so'm
2. Mehrobdan chayon – Abdulla Qodiriy (1929) – 40000 so'm
...
```

**Qo'shimcha (ixtiyoriy, +ball):** Barcha kitoblarning umumiy narxini va o'rtacha narxini hisoblab chiqaring.

## 4-masala. JSON faylni yangilash (o'qish → o'zgartirish → qayta yozish) o'rtacha

Bu masala 5-loyihaning eng muhim qismiga tayyorgarlik. Diqqat bilan bajaring.

### Shart

3-masaladagi `books.json` faylidan foydalaning.

### Bajarish kerak:

- `addBook(book Book) error` — faylni o'qiydi, slice oxiriga **yangi kitob qo'shadi** va faylni qayta yozadi.  
Diqqat: eski ma'lumotlar o'chib ketmasin! Avval o'qish → keyin `append` → keyin yozish.
- Yangi kitobning `ID` si avtomatik hisoblansin: mavjud eng katta ID + 1.
- `updatePrice(id int, newPrice int) error` — berilgan ID'li kitobning narxini o'zgartiradi va faylga saqlaydi.  
Agar bunday ID topilmasa — `fmt.Errorf("book with id %d not found", id)` qaytarsin.
- `deleteBook(id int) error` — berilgan ID'li kitobni o'chiradi va faylga saqlaydi.
- `main` da uchalasini ham chaqirib sinab ko'ring va har safar fayl ichini tekshiring.

**Maslahat — faylni qayta yozish:** `os.Create` faylni **butunlay tozalab** qaytadan yozadi — bu bizga aynan kerak. Sodda variant:

```
data, err := json.MarshalIndent(books, "", " ")
if err != nil {
 return err
}
return os.WriteFile("books.json", data, 0644)
```

**Tez-tez uchraydigan xato:** faylni o'qimasdan turib to'g'ridan-to'g'ri yozib yuborish. U holda faylda faqat **bitta** yangi kitob qoladi, qolgani yo'qoladi.

## 5-masala. LOYIHA: "Online Do'kon" (darsdagi loyihaning to'liq varianti)

asosiy vazifa

### Loyihaning maqsadi

Darsda biz boshlagan loyihani **to'liq ishlaydigan holatga** keltirasiz. Dastur konsol (terminal) orqali ishlaydi va quyidagini qiladi:

1. Foydalanuvchi **login** va **parol** kiritadi.
2. To'g'ri bo'lsa — **MENU** ochiladi.
3. Menyudan **Products** tanlansa — mavjud mahsulotlar ro'yxati chiqadi.
4. Foydalanuvchi mahsulot raqamini va **nechta olishini** tanlaydi.
5. `orders.json` fayliga **yangi order** qo'shiladi.
6. `products.json` faylida o'sha mahsulotning **quantity** si kamayadi.
7. Yo'l-yo'lakay yuz bergan **barcha xatolar** `errors.log` **fayliga** yozib boriladi.

### Fayl tuzilishi

```
vazifa5/
├── main.go
├── users.json # foydalanuvchilar ro'yxati
├── products.json # mahsulotlar
├── orders.json # buyurtmalar (dastur o'zi to'ldiradi)
└── errors.log # xatolar jurnali (dastur o'zi to'ldiradi)
```

## 1) Kerakli structlar

```
type Address struct {
 Name string `json:"name"` // shahar
 Target string `json:"target"` // ko'cha / mo'ljal
}

type User struct {
 ID int `json:"id"`
 Name string `json:"name"`
 Login string `json:"login"`
 Password string `json:"password"`
 Balance int `json:"balance"`
 Address Address `json:"address"`
}

type Product struct {
 ID int `json:"id"`
 Title string `json:"title"`
 Price int `json:"price"`
 Quantity int `json:"qty"`
 IsActive bool `json:"is_active"`
}

type OrderItem struct {
 ProductID int `json:"product_id"`
 Title string `json:"title"`
 Price int `json:"price"`
 Count int `json:"count"` // nechta olindi
 Sum int `json:"sum"` // Price * Count
}

type Order struct {
 ID int `json:"id"`
 UserLogin string `json:"user_login"`
 Items []OrderItem `json:"items"`
 Total int `json:"total"`
 CreatedAt time.Time `json:"created_at"`
}
```

### Nega Order ichida butun User emas, faqat UserLogin?

Chunki buyurtma faylida parolni saqlash **xavfli**, va bir xil ma'lumotni ikki joyda saqlash noto'g'ri. Buni kod ichida komment qilib izohlab qo'ying.

## 2) Boshlang'ich ma'lumotlar

`users.json` faylini **qo'lda** yarating, kamida 2 ta foydalanuvchi bo'lsin:

```
[
 {
 "id": 1,
 "name": "Umidjon",
 "login": "umidjon",
 "password": "12345",
 "balance": 5000,
 "address": { "name": "Samarqand", "target": "Registon" }
 },
 {
 "id": 2,
 "name": "Ali",
 "login": "ali",
 "password": "qwerty",
 "balance": 2000,
 "address": { "name": "Toshkent", "target": "Chilonzor" }
 }
]
```

`products.json` — kamida **5 ta** mahsulot bo'lsin. Agar fayl mavjud bo'lmasa, dastur o'zi **seed** qilib (boshlang'ich ma'lumot yozib) yaratsin.

```
[
 { "id": 1, "title": "Iphone 15", "price": 1200, "qty": 12, "is_active": true },
 { "id": 2, "title": "Samsung S23", "price": 1000, "qty": 5, "is_active": true },
 { "id": 3, "title": "AirPods Pro", "price": 250, "qty": 30, "is_active": true },
 { "id": 4, "title": "MacBook Air", "price": 1500, "qty": 3, "is_active": true },
 { "id": 5, "title": "Xiaomi 13", "price": 700, "qty": 0, "is_active": false }
]
```

`orders.json` — dastur birinchi marta ishga tushganda mavjud bo'lmasa, ichida `[]` yozilgan holda yaratilsin.



### 3) Yozilishi kerak bo'lgan funksiyalar

#### a) Fayl bilan ishlash (storage qatlami)

| Funksiya                                       | Vazifasi                                           |
|------------------------------------------------|----------------------------------------------------|
| <code>loadUsers() ([]User, error)</code>       | <code>users.json</code> ni o'qib slice qaytaradi   |
| <code>loadProducts() ([]Product, error)</code> | <code>products.json</code> ni o'qiydi              |
| <code>saveProducts(p []Product) error</code>   | <code>products.json</code> ni qayta yozadi         |
| <code>loadOrders() ([]Order, error)</code>     | <code>orders.json</code> ni o'qiydi                |
| <code>saveOrders(o []Order) error</code>       | <code>orders.json</code> ni qayta yozadi           |
| <code>seedProducts() error</code>              | fayl yo'q bo'lsa boshlang'ich mahsulotlarni yozadi |

#### b) Xatolarni log qilish — majburiy qism

```
func logError(place string, err error) {
 // errors.log ni "qo'shib yozish" (append) rejimida ochamiz
 file, ferr := os.OpenFile("errors.log",
 os.O_APPEND|os.O_CREATE|os.O_WRONLY, 0644)
 if ferr != nil {
 fmt.Println("could not open errors.log:", ferr)
 return
 }
 defer file.Close()

 line := fmt.Sprintf("[ERROR] %s | %s | %v\n",
 time.Now().Format("2006-01-02 15:04:05"), place, err)

 if _, werr := file.Write([]byte(line)); werr != nil {
 fmt.Println("could not write log:", werr)
 }
}
```

##### Diqqat — `os.Create` emas, `os.OpenFile + O_APPEND` !

`os.Create` ishlatsangiz, har safar eski loglar o'chib ketadi. Bizga esa xatolar **ketma-ket to'planib borishi** kerak.

**Talab:** har bir `err != nil` holatida `logError("funksiya nomi", err)` chaqirilsin. Foydalanuvchiga esa ekranda **sodda** xabar chiqsin (texnik tafsilot log faylda qolsin).

#### errors.log dagi natija taxminan shunday bo'ladi:

```
[ERROR] 2026-08-13 14:22:01 | loadProducts | open products.json: no such file or
directory
[ERROR] 2026-08-13 14:23:15 | login | wrong password for login: ali
[ERROR] 2026-08-13 14:25:40 | buyProduct | not enough stock: Xiaomi 13 (qoldiq 0,
so'ralgan 2)
```

#### c) Login qismi

```
func login(users []User) (*User, error)
```

- Foydalanuvchidan login va parol so'raydi.
- Slice ichidan mos foydalanuvchini qidiradi.
- Topilsa — o'sha foydalanuvchini qaytaradi.
- Topilmasa yoki parol xato bo'lsa — **xato qaytaradi** va `errors.log` ga yoziladi.
- Jami **3 marta** urinish beriladi. 3 martadan keyin dastur to'xtaydi: `"Juda ko'p urinish. Dastur to'xtatildi."`

**Darsdagi koddan farqi:** darsda `checkLogin` o'zini o'zi chaqirar edi (rekursiya) va urinishlar soni cheksiz edi. Endi buni **oddiy for sikli** bilan, 3 ta urinish chegarasi bilan yozing. Bu ancha to'g'riroq yechim.

#### d) Menu

```
===== MENU =====
1. Products
2. My orders
3. Profile
4. Exit
=====
Tanlang:
```

- Menu **tsikl ichida** ishlasin — bitta amaldan keyin dastur yopilib qolmasin, yana menyuga qaytsin.
- Faqat `4. Exit` tanlanganda dastur to'xtasin.
- Noto'g'ri raqam kiritilsa: `"Bunday variant yo'q, qaytadan tanlang"`.

#### e) Mahsulotlarni ko'rsatish

```
func printProducts(products []Product)
```

- Faqat `IsActive == true` **va** `Quantity > 0` bo'lgan mahsulotlar sotuvda deb ko'rsatilsin.
- Qolganlari `(sotuvda yo'q)` deb belgilansin.

```
--- MAHSULOTLAR ---
1. Iphone 15 | 1200$ | qoldiq: 12
2. Samsung S23 | 1000$ | qoldiq: 5
3. AirPods Pro | 250$ | qoldiq: 30
4. MacBook Air | 1500$ | qoldiq: 3
5. Xiaomi 13 | 700$ | (sotuvda yo'q)
```

#### f) Sotib olish — loyihaning yuragi

```
func buyProduct(user *User, productID int, count int) error
```

Bu funksiya quyidagi qadamlarni **shu tartibda** bajaradi:

1. `loadProducts()` — mahsulotlarni o'qish.
2. Berilgan `productID` bo'yicha mahsulotni topish. Topilmasa → xato.
3. **Tekshiruvlar:**
  - `count <= 0` → `"count must be positive"`

- `!IsActive` → "product is not active"
- `Quantity < count` → "not enough stock"
- `user.Balance < price*count` → "insufficient balance"

4. Mahsulot `Quantity` sini `count` ga **kamaytirish**.
5. `saveProducts()` — o'zgargan ro'yxatni faylga qaytarib yozish.
6. `loadOrders()` → yangi `Order` tuzish → `append` → `saveOrders()`.
7. Yangi order `ID` si = mavjud eng katta `ID` + 1 (fayl bo'sh bo'lsa 1).
8. `CreatedAt` = `time.Now()`.
9. Foydalanuvchi balansidan pul yechish (`user.Balance -= total`).
10. Muvaffaqiyatli bo'lsa ekranga chek chiqarish.

### Muvaffaqiyatli xarid natijasi:

```
✓ Buyurtma qabul qilindi!
 Order ID : 3
 Mahsulot : Iphone 15 x 2
 Summa : 2400$
 Qoldiq balans: 2600$
 Manzil : Samarqand, Registon
```

### orders.json fayli natijasi:

```
[
 {
 "id": 1,
 "user_login": "umidjon",
 "items": [
 {
 "product_id": 1,
 "title": "Iphone 15",
 "price": 1200,
 "count": 2,
 "sum": 2400
 }
],
 "total": 2400,
 "created_at": "2026-08-13T14:30:12.123456+05:00"
 }
]
```

**Eng muhim tekshiruv:** xariddan keyin `products.json` ni oching. `Iphone 15` ning `qty` si **12 dan 10 ga** tushgan bo'lishi kerak. Dasturni qayta ishga tushirganda ham yangi qoldiq ko'rinsin — ya'ni ma'lumot faylda haqiqatan saqlangan bo'lsin.

### g) "My orders" bo'limi

- `orders.json` dan **faqat shu foydalanuvchining** buyurtmalari chiqarilsin (`UserLogin` bo'yicha filtrlash).
- Har bir buyurtma sanasi bilan ko'rsatilsin: `CreatedAt.Format("2006-01-02 15:04")`.
- Oxirida **umumiy sarflangan summa** chiqarilsin.
- Buyurtma yo'q bo'lsa: "Sizda hali buyurtmalar yo'q".

## h) "Profile" bo'limi

Foydalanuvchi ismi, logini, balansi va manzilini chiqarsin. **Parol chiqmasin!**

### 4) Baholash mezonlari

| #           | Talab                                                                        | Ball       |
|-------------|------------------------------------------------------------------------------|------------|
| 1           | Structlar va JSON taglar to'g'ri yozilgan                                    | 10         |
| 2           | Login 3 urinish bilan, <code>for</code> sikli orqali ishlaydi                | 10         |
| 3           | Menyu tsikl ichida, Exit'gacha yopilmaydi                                    | 10         |
| 4           | Mahsulotlar ro'yxati fayldan o'qib chiqariladi                               | 10         |
| 5           | <code>orders.json</code> ga yangi order <b>qo'shiladi</b> (eskisi o'chmaydi) | 20         |
| 6           | <code>products.json</code> da qoldiq to'g'ri kamayadi va saqlanadi           | 20         |
| 7           | Barcha xatolar <code>errors.log</code> ga vaqt bilan yoziladi                | 15         |
| 8           | Kod toza, funksiyalarga bo'lingan, <code>gofmt</code> qilingan               | 5          |
| <b>JAMI</b> |                                                                              | <b>100</b> |

### 5) Topshirishdan oldin o'zingizni tekshiring

- ☐ Noto'g'ri parol kiritdim — `errors.log` ga yozildi va 3 urinishdan keyin dastur to'xtadi.
- ☐ Mahsulot sotib oldim — `orders.json` da yangi yozuv paydo bo'ldi.
- ☐ Ikkinchi marta sotib oldim — **eski buyurtma o'chib ketmadi**, ikkitasi ham turibdi.
- ☐ `products.json` dagi `qty` kamaydi va dasturni qayta ochganda ham kam turibdi.
- ☐ Qoldiqdan ko'p miqdorda so'radim — dastur **panic bermadi**, tushunarli xabar berdi.
- ☐ Balansdan qimmat mahsulot oldim — xarid amalga oshmadi.
- ☐ `products.json` ni o'chirib tashladim — dastur qulab tushmadi, xatoni log qildi.
- ☐ Profil bo'limida parol ko'rinmayapti.
- ☐ `go vet ./...` hech qanday ogohlantirish bermadi.

### 6) Qo'shimcha vazifalar (ixtiyoriy, +ball)

- **+5:** Bir buyurtmada **bir nechta** mahsulot sotib olish (savat / cart) — `Items` slice shuning uchun tuzilgan.
- **+5:** Mahsulotlarni narx bo'yicha saralab chiqarish ( `sort.Slice` ).
- **+5:** Nomi bo'yicha mahsulot qidirish ( `strings.Contains` + `strings.ToLower` ).
- **+5:** Xaridan keyin foydalanuvchi balansini ham `users.json` ga saqlab qo'yish.
- **+10:** Structlarga **method** yozish, masalan:

```
func (p *Product) Decrease(count int) error {
 if p.Quantity < count {
 return fmt.Errorf("not enough stock: %s", p.Title)
 }
 p.Quantity -= count
 return nil
}

func (o *Order) CalcTotal() {
 total := 0
 for _, item := range o.Items {
 total += item.Sum
 }
 o.Total = total
}

func (u User) String() string {
 return fmt.Sprintf("%s (%s) — balans: %d$", u.Name, u.Login, u.Balance)
}
```

### Ishni qaerdan boshlash kerak?

1) Avval structlarni yozing. 2) Keyin `load...` / `save...` funksiyalarini yozib, faqat fayl o'qish-yozishni sinang. 3) Keyin `logError`. 4) Keyin `login`. 5) Keyin `menu`. 6) Eng oxirida `buyProduct`. Hammasini birdan yozmang — **har bir qadamdan keyin ishga tushirib ko'ring.**

---

**Muvaffaqiyat tilayman!** 🚀 Savol tug'ilsa — avval xatoni `errors.log` dan o'qing, keyin so'rang.